

MANGAL データ索引 設計仕様 v2 —— 項目定義(レビュー用)

作成: 2026-06-15(改訂) / 目的: 索引を整備する前に「何を・どこから・どう入れるか」を確定する。実装はこの承認後。改訂点: サイト用を“画面ごと”に3分割(検索S1 / 一覧S2 / ホームS3)。

0. 一行サマリ

公開データ(data/manga.v2)を真とし、開発用1本(SQLiteマスター)とサイト用は画面ごとに3本(検索S1 / 一覧S2 / ホームS3)の計4本を promote毎に再生成する。各画面は自分の分だけ遅延ロードし、どの画面も他画面のデータを背負わない。これで「検索が毎回遅い」「トップが全DB送信」「何が欠けているか即答できない」を一気に根治する。

1. 背景と原則

- 問題: 66,582本のYAML(data/manga.v2)を毎回走査=遅い。さらにトップ/検索が全作品オブジェクトをpropsで同梱(数十MB)。
- 既存資産: ① .cache/v2-index.json(69,506作・work級・項目少・JSON線形)。② .cache/db-v2.sqlite(種2の生派生層=166,441 series・公開前で別物)。→ どちらも目的に合わない。
- 原則1 真の源: 公開セット=data/manga.v2(promote後)。実際に表示する姿と一致。
- 原則2 索引はキャッシュ: manga.v2から再生成可。git追跡はレシピ(スクリプト)だけ。
- 原則3 我々用とサイト用は別物: 監査用は“全部入り”、サイト用は“最小”。要件が逆なので混ぜない。
- 原則4 サイト用は画面ごと: 検索・一覧・ホームは必要な列が違う→1枚にせず用途別に割ると1画面の転送が最小。
- 原則5 再導出可能は焼かない: 価格/アフィリエイト等は楽天キャッシュからjoin=索引には最小限/フラグのみ。

2. 成果物(4本・用途別)

成果物	形式 / 置き場	いつ読む	役割 / サイズ目安(本番66k)
A. マスター	SQLite .cache/mangal-index.sqlite	開発時のみ (配信しない)	全項目+FTS+被覆。監査・収穫・将来Worker。~150-250MB
S1. 検索索引	gz JSON(かなシャード) out/idx/search/*.json.gz	検索ボックスを 触った時だけ	題/かな/ローマ字/著者でマッチ。~1.5-2MB(gz)
S2. 一覧索引	gz JSON out/idx/list.json.gz	一覧表(/list)を 開いた時だけ	絞り込み・ソート列。~2.5MB(gz)
S3. ホームリスト	小JSON out/idx/home.json	ホーム初回	各セクションの“描くだけ”完成リスト。~数十KB

S1/S2/S3 はすべて A から機械生成する派生物(源は1本)。誰も全部は読まないのが要点。

3. マスターA のテーブル構成(粒度)

テーブル	粒度	主な用途
works	1作(=1公開ページ)1行	作品単位の被覆・ソート・S1/S2/S3の元
volumes	1巻1行	ISBN/表紙/発売日/価格の被覆・ストアリンク・収穫対象抽出
authors / work_authors	著者1行 / 作品×著者	著者検索・50音索引・原作/作画分離
works_fts (FTS5)	works対応の全文索引	題・かな・ローマ字・別名・著者名の即時部分一致(開発側)
coverage (ビュー)	集計1行	『何が入って何が欠けているか』を1クエリで

4. 項目定義 ―― これが本体

型は SQLite 表記。源『v2』=data/manga.v2、『算出』=ビルド時計算、『楽天』=.cache/rakuten-isbn.jsonl から join。

4-1. works(作品)

項目	型	源	用途・備考
slug	TEXT PK	v2	URL・主キー
title / title_kana / title_romaji	TEXT	v2	表示 / 50音ソート・かな検索 / ローマ字検索
subtitle	TEXT	v2	表示(副題)
alt_title_en	TEXT	v2.alternative_title s.en	公式英題=検索・リダイレクト(slugには使わない方針)
alt_titles_json	JSON	v2.alternative_title s	多言語別名
synonyms_json	JSON	v2.synonyms	検索recall。※日本語重複が多く表示には出さない(検索専用)
demographic / status	TEXT	v2	絞り込み(少年/青年… / 連載・完結・休載)
publisher_key / publishers_json	TEXT/JSON	v2	主出版社 / 全出版社(多社作品)
magazine_key	TEXT	v2.magazine	掲載誌で絞り込み
genres_json	JSON	v2.genres	ファセット絞り込み(master32キー)
genres_provisional	BOOL	v2	AI推定ジャンルの低信頼マーク
genres_anilist_json / tags_json	JSON	v2	裏取り/補助 / 要素(あらすじ要素)検索
year_started / year_ended	INT	v2	年代絞り込み・タイムライン
first_volume_date	TEXT	算出	standard版1巻の最小発売日=既定ソートキー
anilist_id	INT	v2	外部リンク・重複検知
wikidata_qid / work_wikidata_qid	TEXT	v2	著者QID / 作品QID(外部リンク)
popularity / score	INT/REAL	v2	人気ソート・ランキング / 評価ソート
adult_us	BOOL	v2	米基準成人=geo出し分け
anime_adapted	BOOL	v2	アニメ化フラグ
catch	TEXT	v2(seed)	キャッチコピー(カード表示)
synopsis_len	INT	算出 len(synopsis)	被覆判定(本文はFTSへ。索引肥大回避で本文は持たない)
volume_count / edition_count	INT	算出	規模
isbn_count / isbn_missing	INT	算出	被覆 / ISBN収獲対象抽出
cover_count / cover_missing / has_cover	INT/BOOL	算出(楽天join)	表紙被覆 / カード表示可否
rep_cover_url	TEXT	算出	代表表紙(1巻優先→表紙ある巻にフォールバック)
search_blob	TEXT	算出	FTSの源(題+かな+ローマ字+英題+別名+著者名を正規化連結)
has_kana/has_romaji/has_synopsis/ has_genres/has_full_isbn/has_anilist/has_work_qid …	BOOL	算出	被覆フラグ群=監査を1クエリ化。欠けの所在が即わかる

4-2. volumes(巻)

項目	型	源	用途・備考
id / work_slug	INT / TEXT FK	算出	主キー / 親作品
edition_type / edition_label / imprint / edition_publisher	TEXT	v2.editions	版種(通常/完全/愛蔵…)・レーベル・版元
number / is_extra	INT/BOOL	v2	巻番号 / 番外
isbn13 / isbn_present	TEXT/BOOL	v2	ISBN / 有無(ISBN欠落監査)
release_date / date_precision	TEXT/TEXT	v2/算出	発売日 / 精度(day=日まで, month=月まで, none)
cover_url / cover_present / cover_source	TEXT/BOOL/TEXT	楽天join	表紙 / 有無 / 出所(rakuten…)。noimageは無扱い
asin	TEXT	v2	Amazon識別(将来PA-API用)
price / affiliate_url	INT/TEXT	楽天join	価格 / 楽天アフィリエイト(収益導線。最小限のみ保持)

4-3. authors / work_authors(著者)

項目	型	源	用途・備考
authors.key	TEXT PK	算出	著者QIDまたは正規化名
authors.name / kana / romaji	TEXT	v2.authors	表示 / 50音索引・かな検索 / ローマ字検索
authors.alt_names_json	JSON	DB mangaka	表記揺れ(検索recall)
authors.work_count / has_adult_credit	INT/BOOL	算出	著者ページ / 成人signal
work_authors.(work_slug, author_key, role)	—	v2	作品×著者×役割(原作/作画/原作者を分離)

4-4. works_fts(全文検索) / coverage(被覆)

- works_fts = SQLite FTS5。search_blob をトークン化し、題・かな・ローマ字・英題・別名・著者名の部分一致を即時検索。かなは正規化(全角/半角・カタカナ/ひらがな・長音)して取りこぼしを減らす。
- coverage = 各項目の充足件数を持つ集計(ビュー or 物理表)。例:総数・kana有・synopsis有・全巻ISBN有・表紙有 を1行で。年代別・人気帯別の欠けも GROUP BY で即出る。

5. サイト用3本の項目定義(S1/S2/S3)

いずれも works から抽出。各画面が要る列だけに絞り、他は持たない=最小。

5-1. S1 検索索引(検索用・最小)

マッチと結果1行の描画に要る分だけ。分野/年/人気すら入れない(結果行に出さないなら不要)。かな頭文字でシャード化(あ/か/さ…/英数/他=約12-15ファイル)。

キー	内容	理由
slug	URL	遷移先
t / k / r	題 / かな / ローマ字	3経路でヒット(べるせるく/ベルセルク/berserk)
a	代表著者名	著者でも引ける

約90バイト/作 → 本番66kでgz ~1.5-2MB。検索時のみロード→IndexedDBにキャッシュ。

5-2. S2 一覧索引(/list の絞り込み・ソート用)

キー	内容	理由
slug / t / a	URL / 題 / 著者	行の表示
d	demographic	分野で絞り込み
g	genres[]	ジャンルで絞り込み
pub / mag	出版社 / 掲載誌	絞り込み
st	status	連載/完結/休載で絞り込み
y / p	年 / popularity	ソート
c	has_cover(0/1)	サムネ可否

/list を開いた時だけロード。検索(S1)とは要る列が違うので別ファイル。

5-3. S3 ホーム完成リスト(索引ですらない)

ホームは検索も絞り込みもしない。各セクションの“描くだけ”の小リストを事前計算して持つ。

セクション(例)	中身	件数目安
popular / new	人気トップ / 新刊	各 30-60
genre/[key]	ジャンル別トップ	各 12-24
sansedai / featured	三世代 / 今週の一冊	日替/週替
(各item)	slug / 題 / 著者 / 代表表紙URL	—

合計数十KB。ホーム初回はこれだけ=全DBを送らない。

6. 画面別ロード方針 —— 誰も全部は読まない

画面	読む物	転送(本番目安)
ホーム /	S3 のみ	数十KB
検索	S1 を追加(初回のみ・以後キャッシュ)	1.5-2MB→0
一覧表 /list	S2 を追加	~2.5MB
詳細 /manga/[slug]	そのページ単体ファイル	そのページ分だけ

今(全DBをpropsで数十MB同梱)→ 後(ホーム数十KB)。本質はアルゴリズムでなく“送りすぎ”の解消。

7. ビルドと更新フロー

- スクリプト scripts/_build-index.py(新規。既存 _build-v2-index.py を発展)。
- 入力 = data/manga.v2/*.yaml(CSafeLoader)+ .cache/rakuten-isbn.jsonl(表紙/価格/アフィリエイト)+ 必要seed。
- 処理 = 正規化(かな/ローマ字)→ A(SQLite: works/volumes/authors/FTS/coverage)を構築 → Aから S1/S2/S3 を機械生成(源は1本)。

- 更新タイミング = promote毎・月次蒸留毎。全再構築(幂等)。目安 数分(66k)。

8. テスト環境(600件)での試作計画 —— まずここから

- 対象 = mangal-preview(600件サブセット)。本番DB・稼働中の表紙収獲に触れない隔離環境。
- 手順 = ① _build-index.py で .preview-data から S1/S2/S3 生成 → ② 既存を壊さないよう新ルート(例 /search-proto)でS1遅延ロード検索を試作・/list を S2 化・ホーム一部を S3 化 → ③ ビルド→Pagesデプロイ→実機確認。
- 検証できる = 検索の即時性 / 遅延ロード / シャード / IndexedDBキャッシュ / かな・ローマ字・英語の取りこぼし無し / 「全DBをpropsで送らない」動作。
- 限界(正直) = 600件だとサイズ削減の“絶対値”は小さい(数十KB→数KB級)。本当の“数十MB→数十KB”は本番66k(R2)で出る。→ テストでは仕組み・UX・正しさを固める。サイズ実測したい時は本番規模の索引ファイルだけ別途previewに置いて計測も可。

9. 保存・永続化の方針

- マスターSQLite = .cache(再生成可能なので非追跡)。消えても manga.v2 から作り直せる。
- 軽量JSON = ビルド時に out/ へ生成し配信(git追跡しない or 小さいので任意)。
- git追跡するのはレシピ(_build-index.py)だけ=原則2/4に合致。

10. 入れないもの(肥大・陳腐化の回避)

- synopsis本文(全文)→ works表には長さ(被覆)だけ。本文はFTSに入れて検索だけ可能に。サイト用(S1/S2/S3)には一切入れない。
- 楽天の生JSON / 表紙画像そのもの → 持たない(URLとフラグのみ)。
- 種2生層(db-v2)の重複保持 → しない。本索引は公開層専用。
- 監査用フィールド(被覆フラグ・isbn欠落数・search_blob・各QID等)→ A だけに置きサイト用には出さない。
- 頻繁に変わる在庫・リアルタイム価格 → 索引には入れず実行時API(将来)。

11. これで“即答”になる例(マスターA)

- ISBN欠落を年代別に: works/volumes を GROUP BY 年代(今日は数分の走査が必要だった)。
- 表紙が無い人気作 上位: WHERE has_cover=0 ORDER BY popularity DESC。
- あらすじ未充填の連載中作: WHERE has_synopsis=0 AND status='ongoing'。
- かな欠落・slug衝突候補・多社作品 等の品質監査も1クエリ。
- 収獲対象(表紙/ISBN欠け)の抽出も SELECT 一発=次の収獲が即組める。

12. 未決点(着手前に確認したいこと)

※「我々用とサイト用を分ける」「サイト用は画面別3分割」は決定済み(本v2に反映)。残る論点:

- (a) FTSのかなトークナイズ: 日本語は分かち書きが必要。N-gram(2-gram)方式にするか(部分一致に強い)。※開発A側のみ。
- (b) S1のシャード粒度: かな頭文字(あ/か/さ…)で12-15分割で良いか、英数/記号始まりの束ね方。
- (c) IndexedDBキャッシュのバージョンニング: ビルドのハッシュで無効化する方式で良いか。
- (d) 著者キー: QID無し著者の名寄せ(同名異人/表記揺れ)をどこまで索引で吸収するか。
- (e) 成人作: adult_us は持つが、サイト用を含める/隠す(geoはWorker側)を最終的にどうするか。

以上(v2)。まずテスト環境(600件)で §8 の試作に着手します。修正点があれば本書を作り直します。